

Shortest Path Algorithms

Dijkstra, Floyd-Warshall, Bellman-Ford



Pre-requisites

- Heap data structure
- Graph data structure





Dijkstra Algorithm

Few basic points about Dijkstra

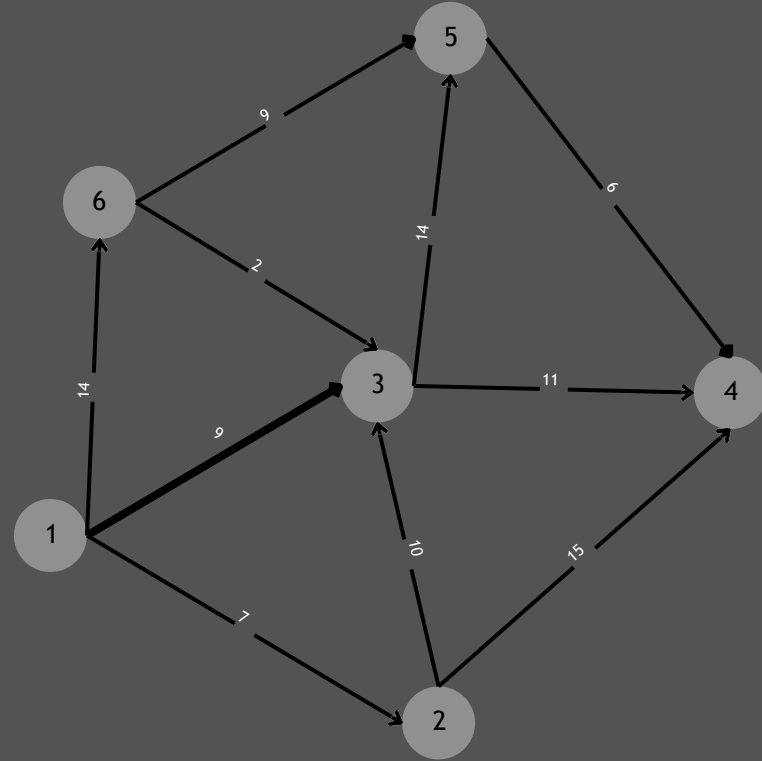
- Dijkstra's algorithm finds the shortest paths from a single source node to all other nodes in a graph.
- Dijkstra's algorithm is greedy, meaning that it makes the locally optimal choice at each step in the hope of finding the globally optimal solution.

Demonstration



min-heap

distance = 0
node = 1

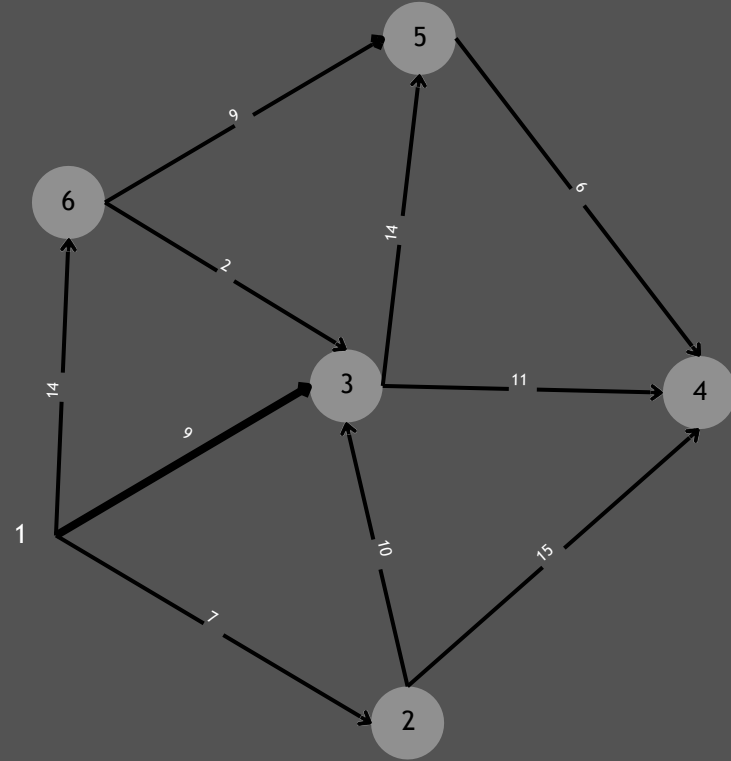


min-heap

current node

distance = 0
node = 1

dist =
{1:0}



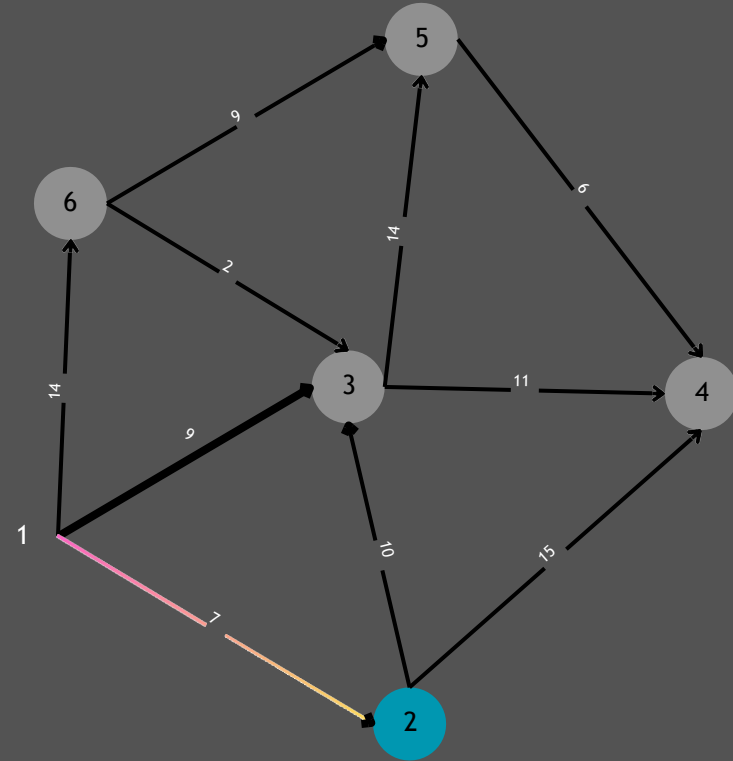
min-heap

distance = 7
node = 2

current node

distance = 0
node = 1

dist =
{1:0}



min-heap

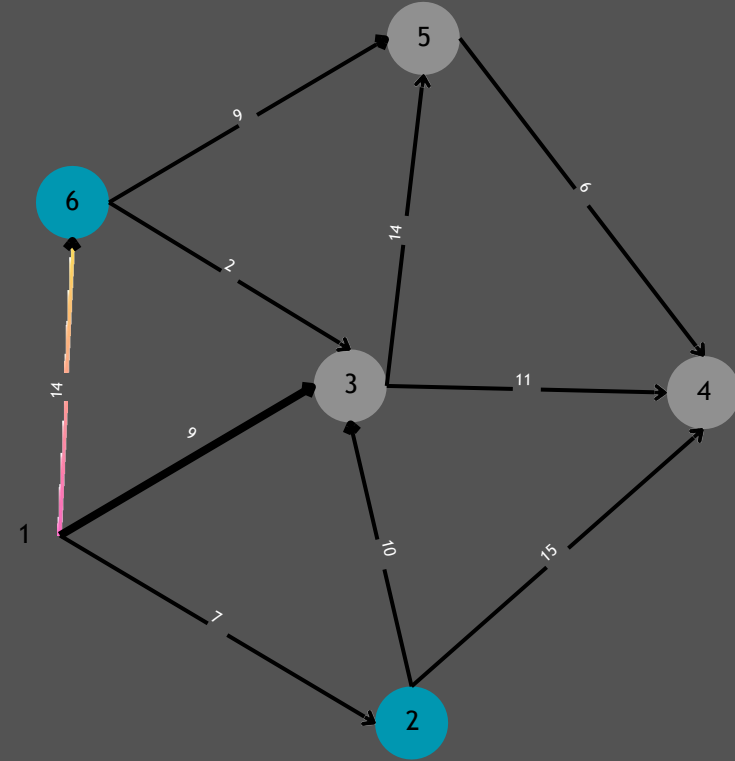
distance = 7
node = 2

distance = 14
node = 6

current node

distance = 0
node = 1

dist =
{1:0}



min-heap

distance = 7
node = 2

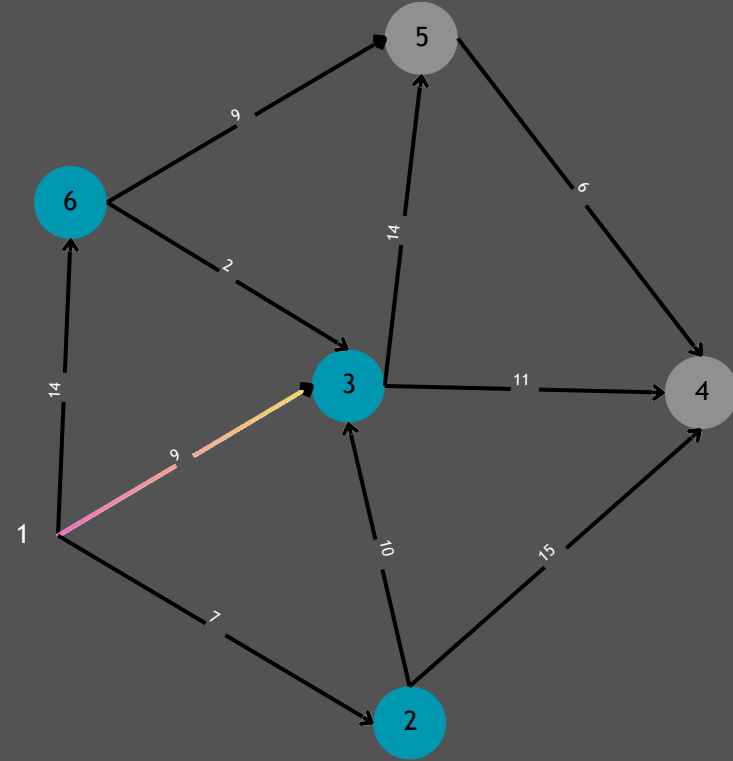
distance = 14
node = 6

distance = 9
node = 3

current node

distance = 0
node = 1

dist =
{1:0}



min-heap

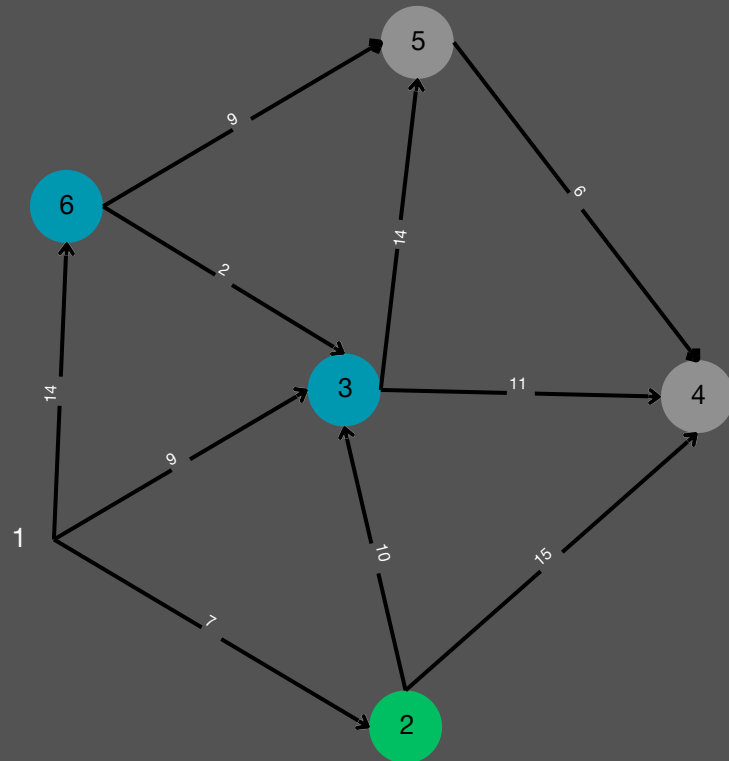
distance = 14
node = 6

distance = 9
node = 3

current node

distance = 7
node = 2

dist = {1:0, 2:7}



min-heap

distance = 14
node = 6

distance = 9
node = 3

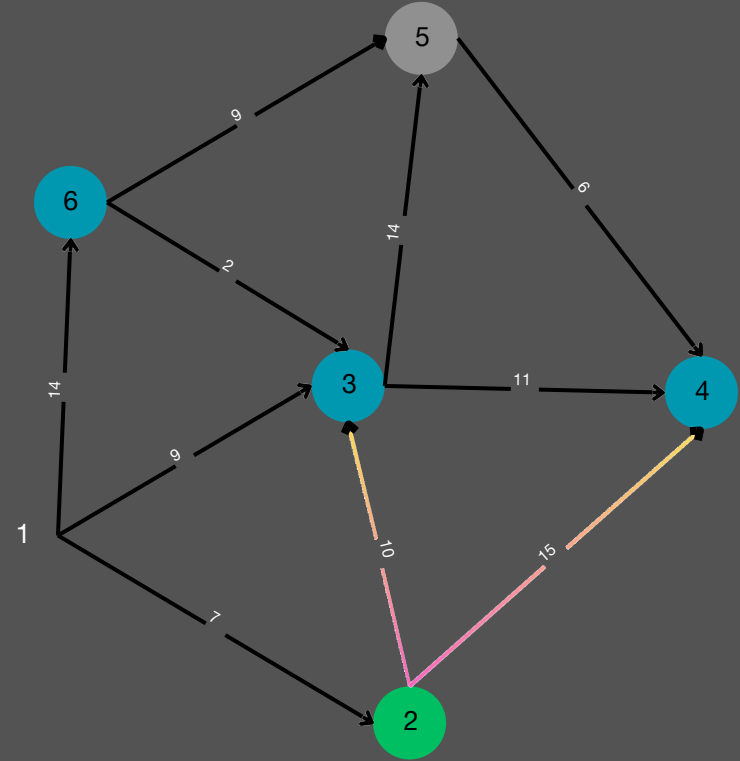
distance = 17, node = 3

distance = 22, node = 4

current node

distance = 7
node = 2

dist = {1:0, 2:7}



min-heap

distance = 14
node = 6

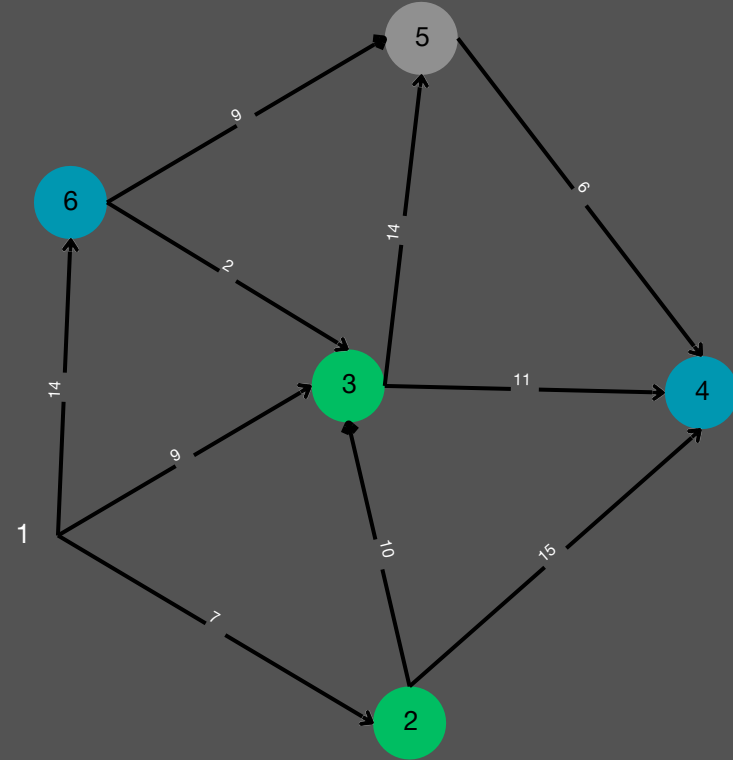
distance = 17, node = 3

distance = 22, node = 4

current node

distance = 9
node = 3

dist = {1:0, 2:7, 3:9}



min-heap

distance = 14
node = 6

distance = 17
node = 3

distance = 22, node = 4

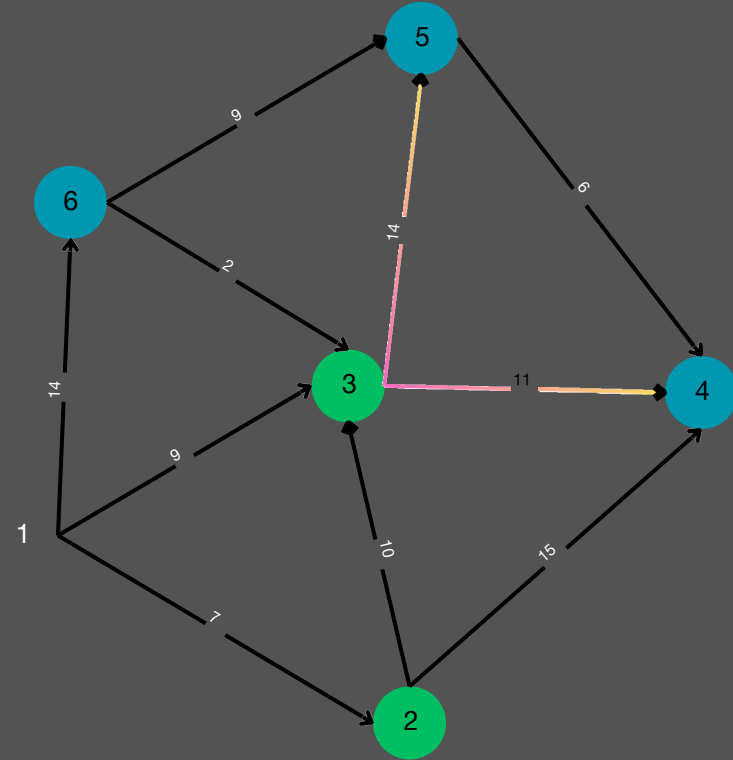
distance = 23, node = 5

distance = 20
node = 4

current node

distance = 9
node = 3

dist = {1:0, 2:7, 3:9}



min-heap

distance = 23
node = 5

distance = 17
node = 3

distance = 22
node = 4

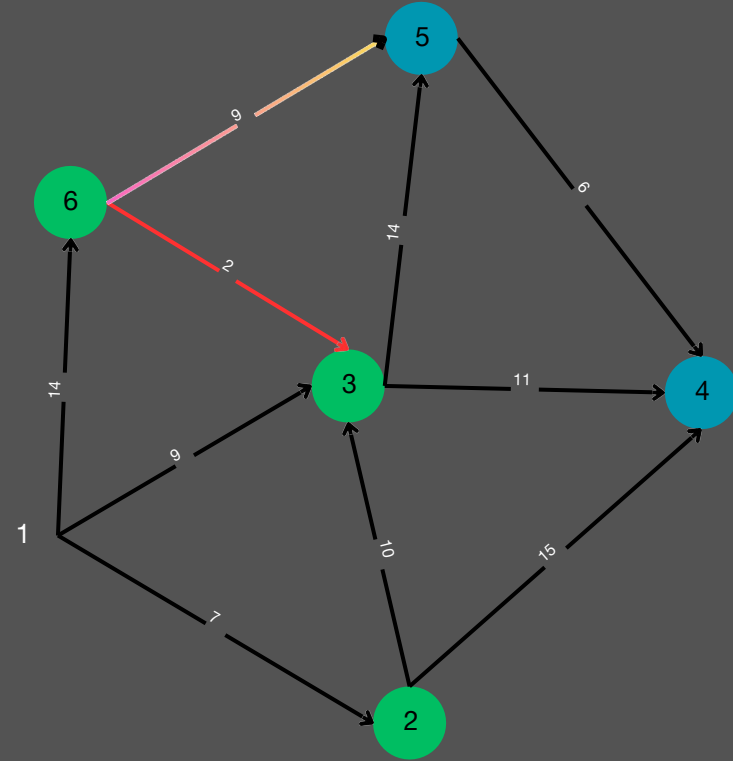
distance = 23
node = 5

distance = 20
node = 4

current node

distance = 14
node = 6

dist = {1:0, 2:7, 3:9, 6:14}



min-heap

distance = 23
node = 5

distance = 22
node = 4

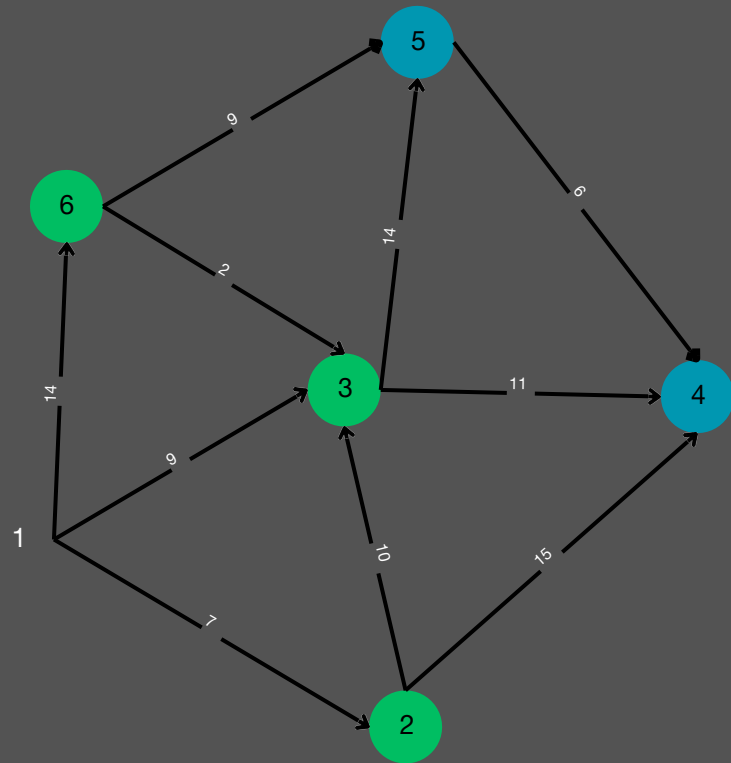
distance = 23
node = 5

distance = 20
node = 4

current node

distance = 17
node = 3

dist = {1:0, 2:7, 3:9, 6:14}



min-heap

distance = 23
node = 5

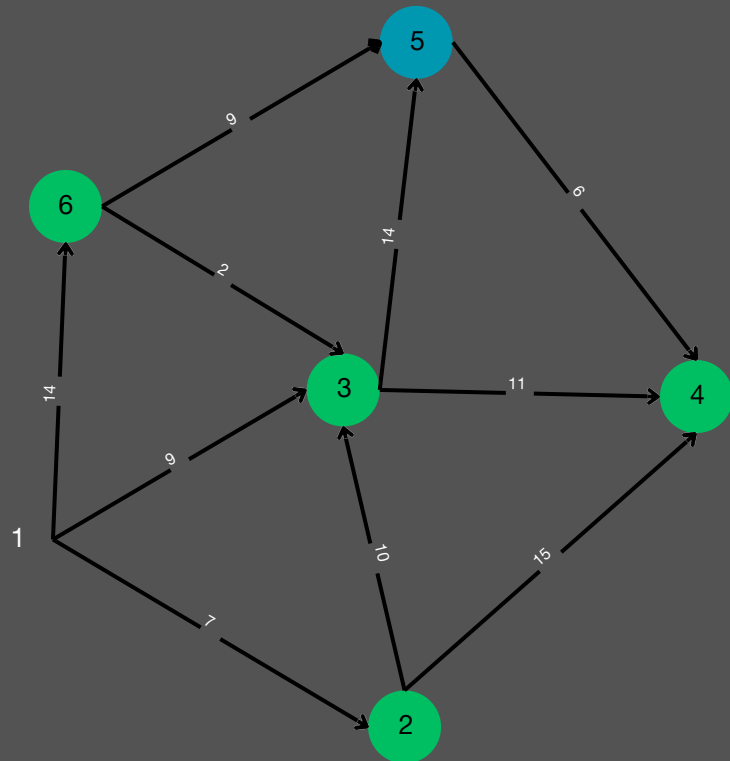
distance =
22
node = 4

distance = 23
node = 5

current node

distance = 20
node = 4

dist = {1:0, 2:7, 3:9, 6:14,
4:20}



min-heap

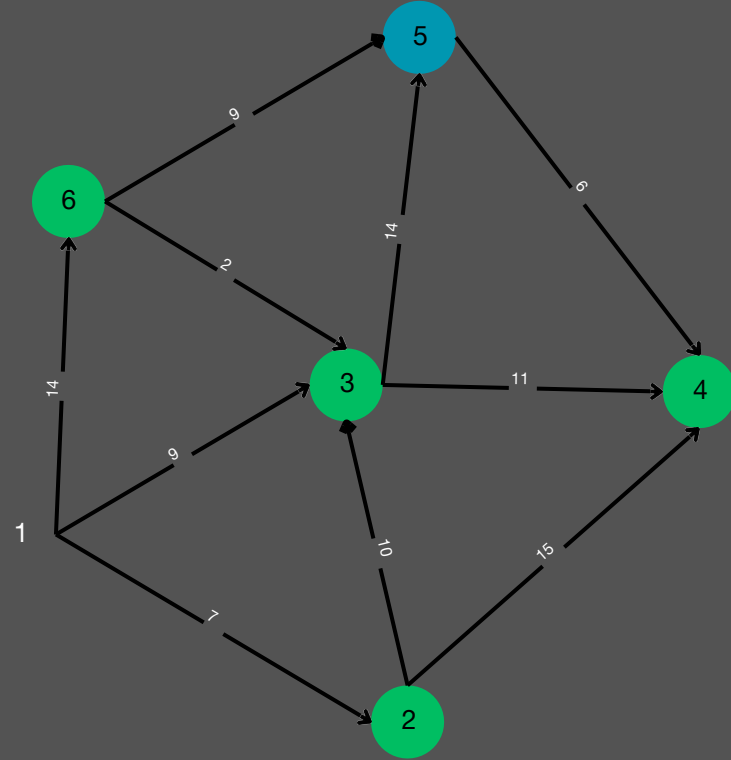
distance = 23
node = 5

distance = 23
node = 5

current node

distance = 22
node = 4

dist = {1:0, 2:7, 3:9,
6:14,
4:20}



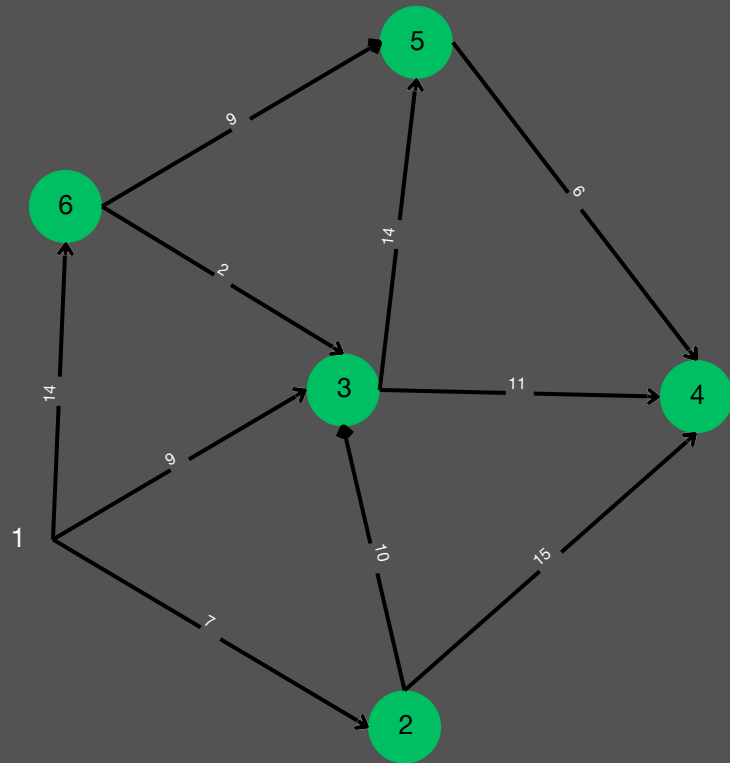
min-heap

distance = 23
node = 5

current node

distance = 23
node = 5

dist = {1:0, 2:7, 3:9, 6:14,
4:20, 5:23}

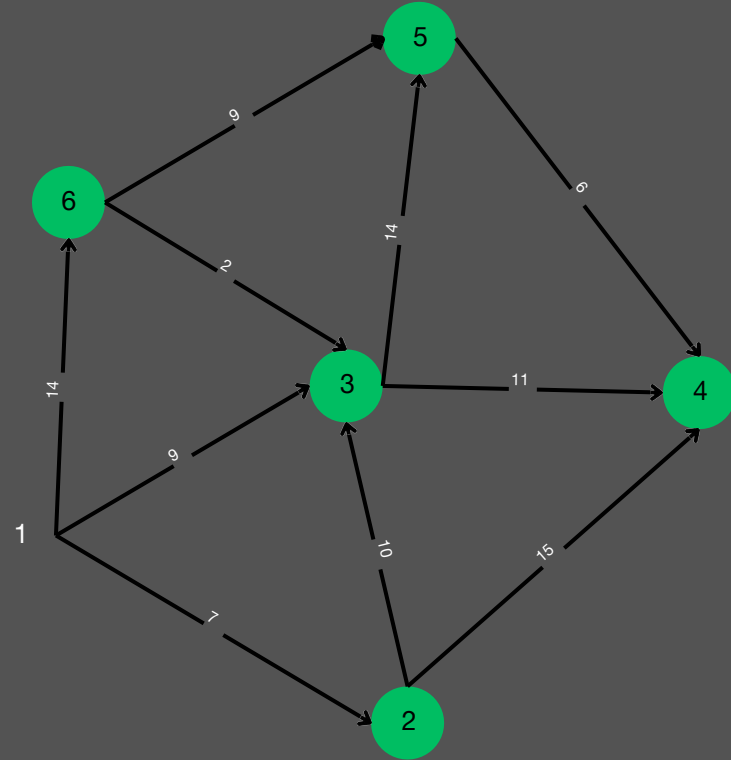


min-heap

current node

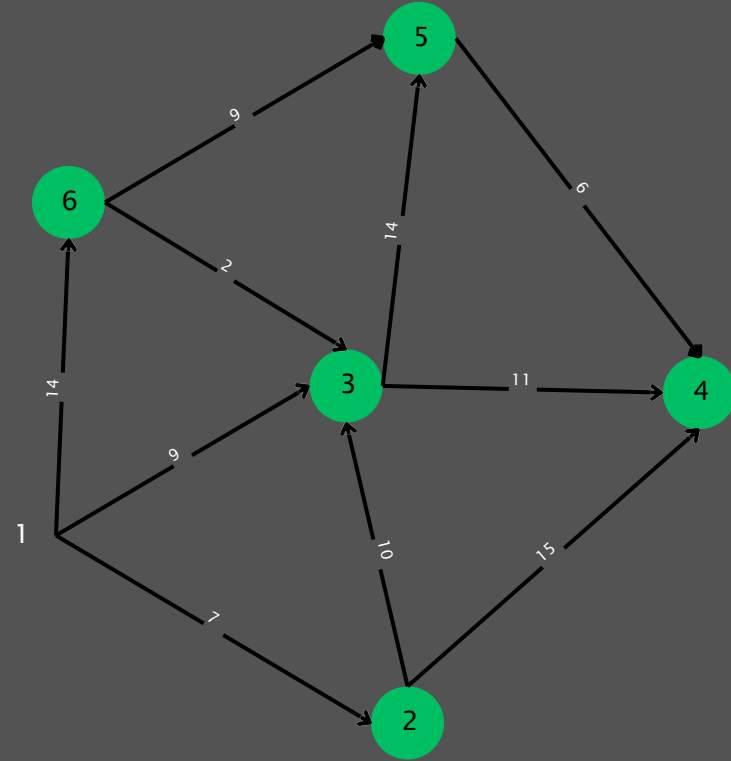
distance = 23
node = 5

dist = {1:0, 2:7, 3:9, 6:14,
4:20, 5:23}



Final Answer

dist = {1:0, 2:7, 3:9, 6:14, 4:20, 5:23}



Can you see any similarities with BFS Algorithm?

Does dijkstra work for unweighted graphs?

Implementation

1. Initialize a **min-heap (priority queue)** and a distances array.
2. Set all distances to **infinity** except for the start vertex ($\text{distances}[\text{start}] = 0$).
3. Add the start vertex to the min-heap with a distance of 0.
4. While the min-heap is **not empty**:
 - a. Extract the vertex with the minimum distance from the min-heap.
 - b. For each neighbor of the current vertex:
 - i. Calculate the total distance to the neighbor through the current vertex.
 - ii. If this total distance is less than the current recorded distance to the neighbor, update the distance and insert the neighbor into the min-heap.
5. After the loop, the distances array contains the **shortest distances from the start vertex** to all other vertices.

```

import heapq

def dijkstra(graph, start_node):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    visited = set()

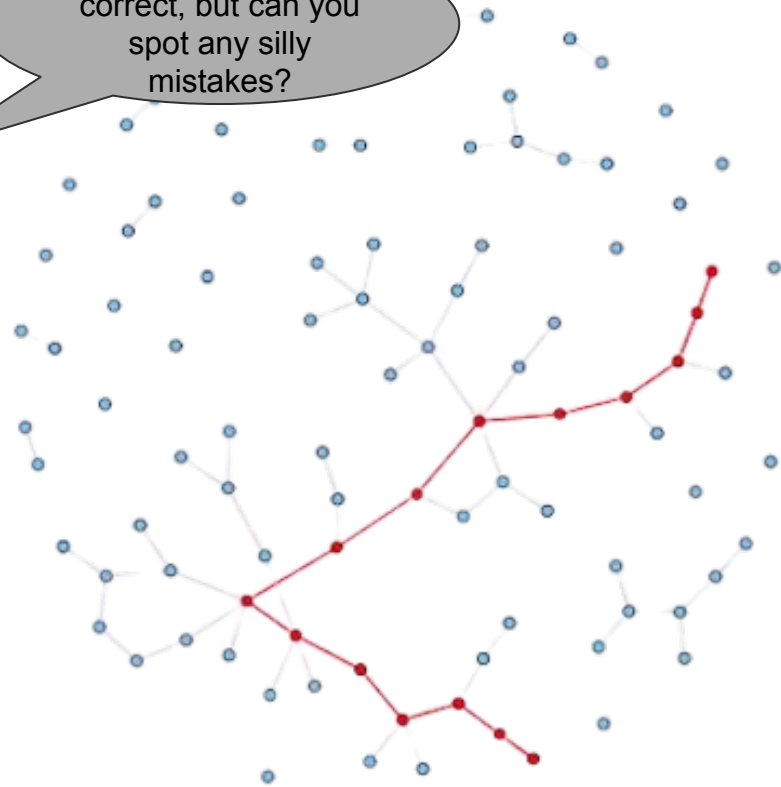
    heap = []
    while heap:
        current_distance = heapq.heappop(heap)
        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(heap, distance)
    return distances

```



It looks like it's correct, but can you spot any silly mistakes?



```

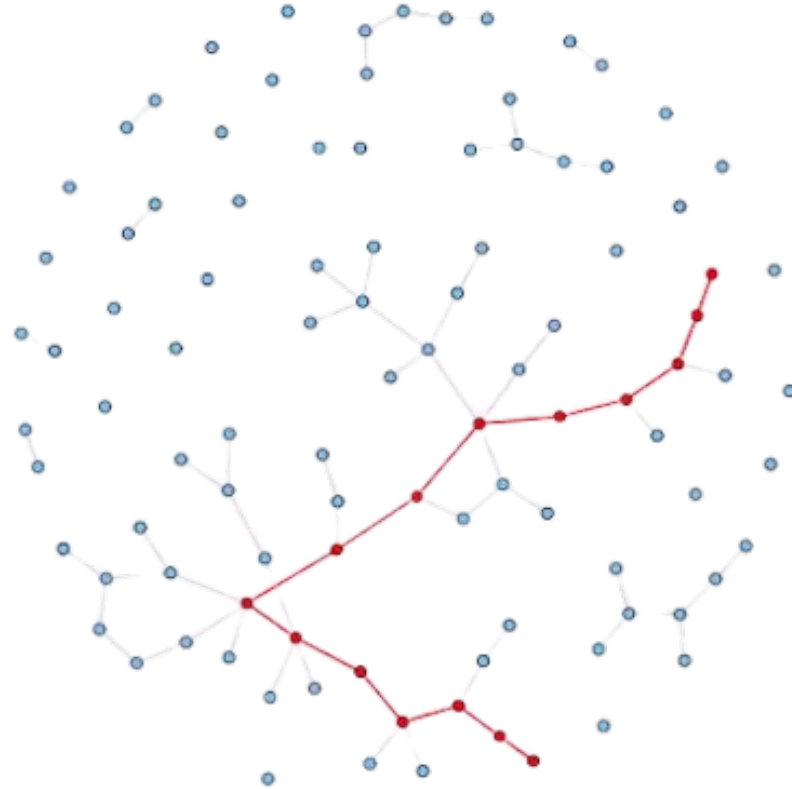
import heapq
def dijkstra(graph, start_node):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    visited = set()

    heap = []
    while heap:
        current_node = heapq.heappop(heap)
        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(heap, neighbor)
    return distances

```

The heap is
empty at first
!



```
import heapq

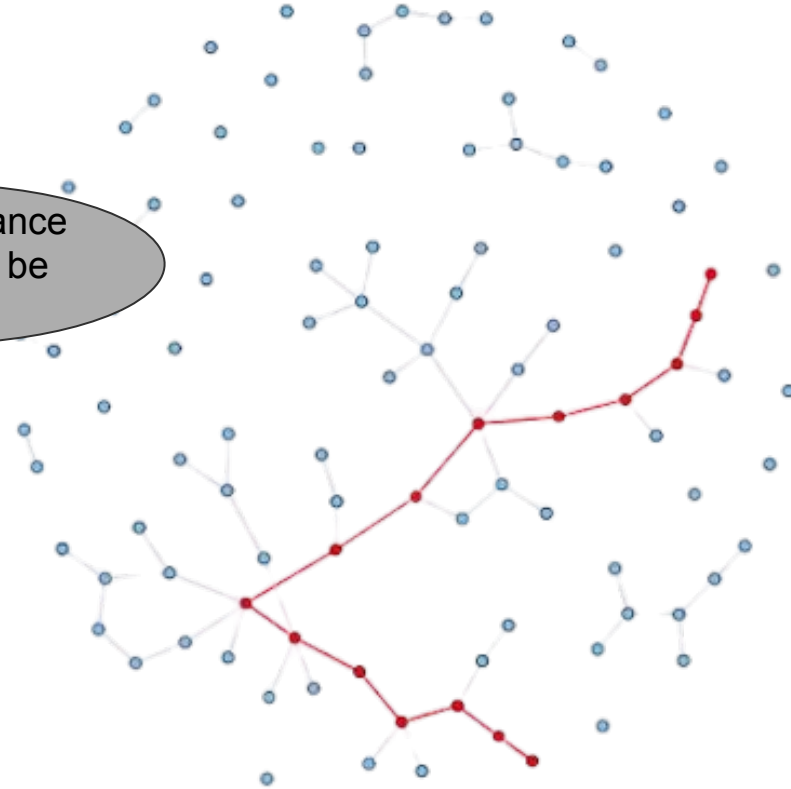
def dijkstra(graph, start_node):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    visited = set()

    heap = [(0, start_node)]
    while heap:
        current_node = heapq.heappop(heap)
        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(heap, neighbor)
    return distances
```



Current_distance
Should also be
popped



```
import heapq

def dijkstra(graph, start_node):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    visited = set()

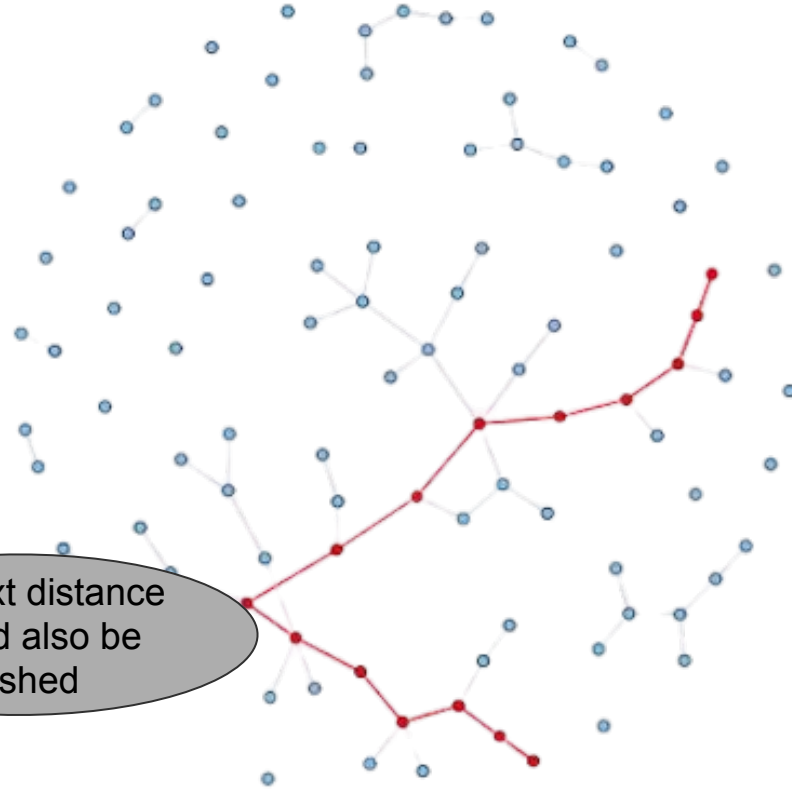
    heap = [(0, start_node)]
    while heap:
        Current_distance, current_node = heapq.heappop(heap)
        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(heap, neighbor)

    return distances
```



The next distance
Should also be
pushed



Working Implementation

```
import heapq

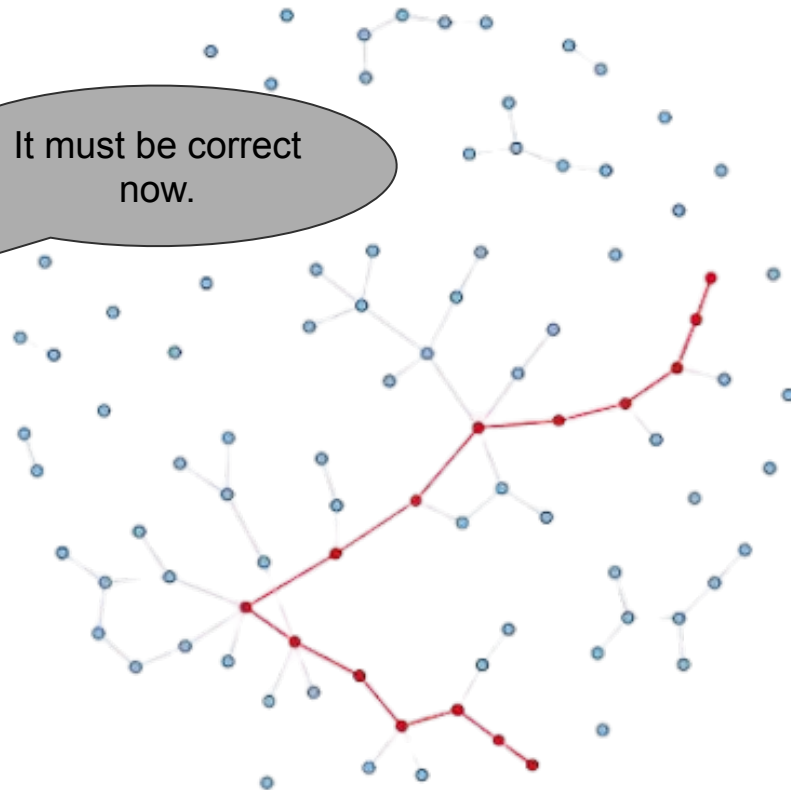
def dijkstra(graph, start_node):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    visited = set()

    heap = [(0, start_node)]
    while heap:
        current_distance, current_node = heapq.heappop(heap)
        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(heap, (distance, neighbor))
    return distances
```



It must be correct now.



Time and space complexity

Time Complexity: $O((V + E) * \log(V))$

Space Complexity: $O(V)$

- **Problem:** How do you Find the shortest sequence of nodes (i.e., the shortest path) from a source node 🏠 to a destination node 🏁 in a graph.

1. Prepare a dictionary *predecessor* to store the predecessor node of each node in the shortest path.
2. Maintain an additional variable *predecessor_node* to store the predecessor node of the current node inside the heap.
3. After the algorithm finishes, reconstruct the shortest path by following the predecessor nodes from the destination node to the source node.

Reconstruction Algorithm:

1. Start with the destination vertex v .
2. Repeatedly take the predecessor of the current vertex until reaching the source vertex s .
3. The reconstructed path '**Path**' will have vertices listed in **reverse order**.

Real world Applications







2. Optimizing computer networks

Floyd-Warshall Algorithm



What If?

What if you wanted to find the **shortest path** between all pairs of vertices in a graph?

Possible solution

Easy: you can run dijkstra for each node.

- * What would be the time complexity?

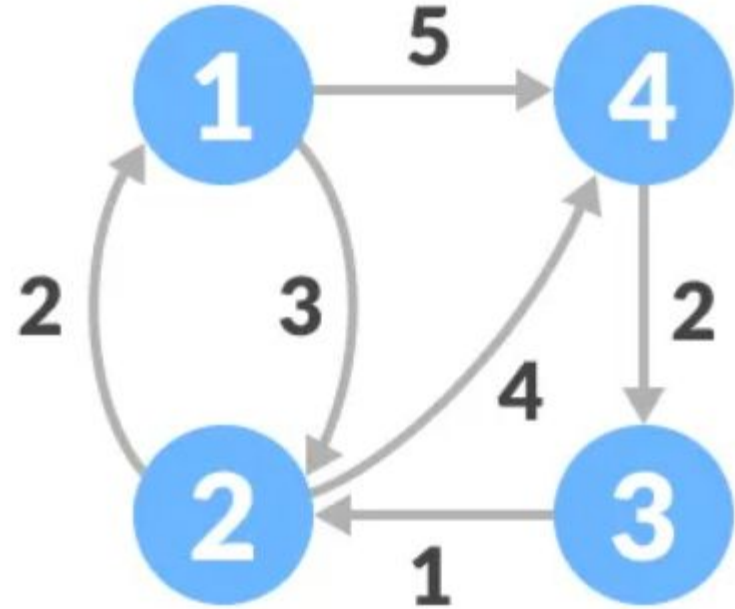
using floyd-warshall algorithm you can do it in just around 4 lines of code.

Basic Ideas of the algorithm

- First we need a matrix that will hold all the shortest distance of the pairs.
- At first the matrix values will be infinity, except for the pairs where there is a direct edge and for pairs with the same node.
- At each iteration, the algorithm considers all possible paths between each pair of vertices, and updates the distance matrix if a shorter path is found.

Example

Let's find shortest distance for all pairs

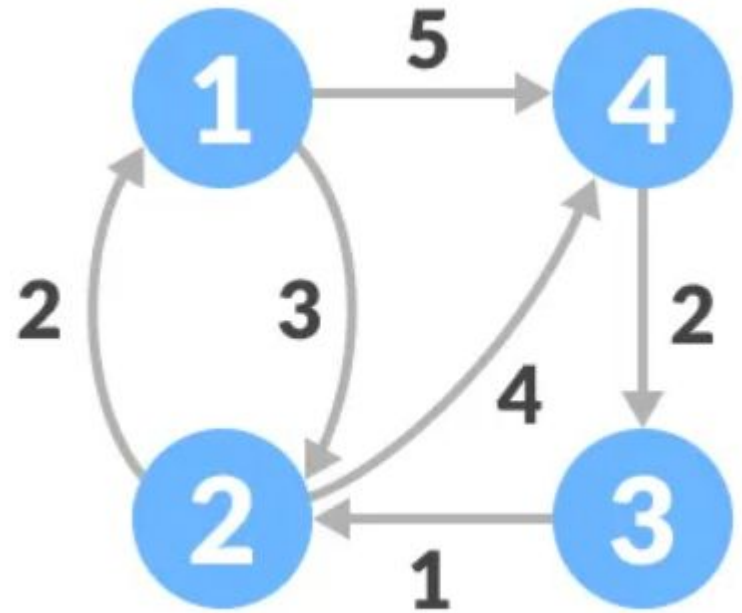


Create a matrix A_0 of dimension $n \times n$ where n is the number of vertices. The row and the column are indexed as i and j respectively. i and j are the vertices of the graph.

Each cell $A[i][j]$ is filled with the distance from the i th vertex to the j th vertex. If there is no path from i th vertex to j th vertex, the cell is left as **infinity**.

$A^0 \Rightarrow$

0	3	∞	5
2	0	∞	4
∞	1	0	∞
∞	∞	2	0



Now, the remaining cells are filled in the following way.

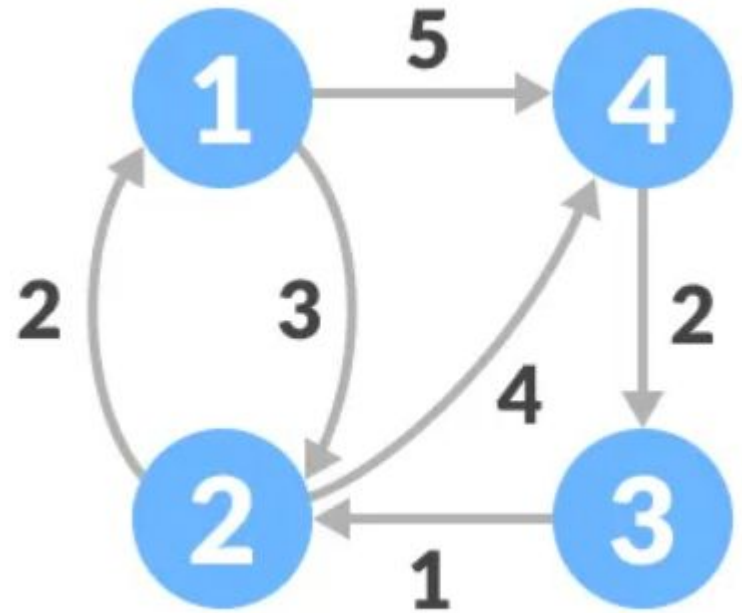
Let k be the **freezing vertex** in the shortest path from source to destination. In this step, k is the first vertex.

$A[i][j]$ is filled with $(A[i][k] + A[k][j])$ if $(A[i][j] > A[i][k] + A[k][j])$.

In this step, k is vertex 1. We calculate the distance from the source vertex to destination vertex through this vertex k .

$A^1 \Rightarrow$

0	3	∞	5
2	0	∞	4
∞	1	0	∞
∞	∞	2	0

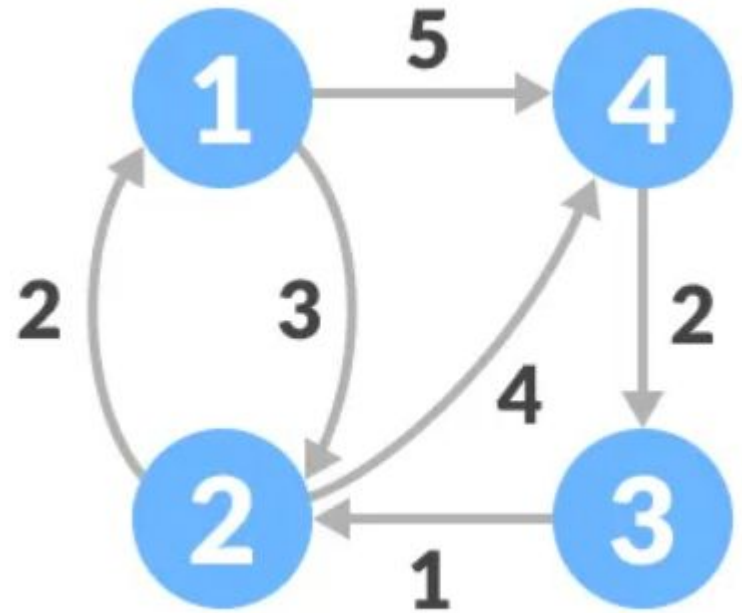


In a similar way, A_2 is created using A_1 . The elements in the second column and the second row are left as they are.

In this step, k is the second vertex (i.e. vertex 2). The remaining steps are the same as in step 2.

$A^2 \Rightarrow$

0	3	∞	5
2	0	∞	4
3	1	0	5
∞	∞	2	0



$$A^3 \Rightarrow \begin{array}{cccc} & 0 & 3 & \infty & 5 \\ & 2 & 0 & \infty & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{array}$$

$$A^4 \Rightarrow \begin{array}{cccc} & 0 & 3 & 7 & 5 \\ & 2 & 0 & 6 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{array}$$

```
dist = [[float('inf')] * n for _ in range(n)]

for i, j, w in edges:
    dist[i][j] = dist[j][i] = w

for i in range(n):
    dist[i][i] = 0

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

Time and Space Complexity

Time Complexity: The Floyd-Warshall algorithm runs in $O(V^3)$ time, where V is the number of vertices in the graph. This is because the algorithm iterates over all vertices and considers all possible pairs of vertices.

Space Complexity: The algorithm requires $O(V^2)$ space to store the distance matrix.

Real-Life Application

The Floyd-Warshall algorithm has various real-life applications, including:

Routing protocols in computer networks: It can be used to find the shortest paths between routers in a network.

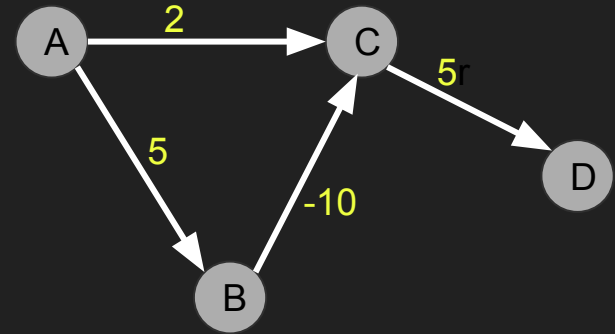
Transportation networks: It can be employed to determine the most efficient routes between locations in a transportation network.

DNA sequence alignment: It can help identify the similarity between two DNA sequences by finding the shortest path of mutations.

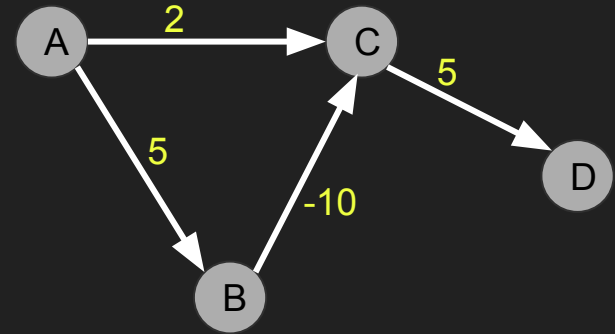
Bellman Ford's Algorithm



What is the shortest path A $\rightarrow$ D using Dijkstra?



Answer :Dijkstra will be saying its 7, while it's actually 0



Dijkstra's and Floyd-Warshall algorithms **don't work** for graphs with negative edges.



Bellman-Ford algorithm is used to find the shortest distance in a graph, even when there are negative edge weights.

- The Bellman-Ford algorithm is a shortest path algorithm used to find the shortest distance from a source vertex to all other vertices in a graph.
- It iteratively relaxes the edges of the graph to update the distance estimates until it finds the shortest paths.
- In each iteration, it relaxes the edges, which means it tries to improve the distance estimates if a shorter path is found.

Relaxation process

- The relaxation process in the Bellman-Ford algorithm compares the current distance estimate of a vertex **with the distance estimate through the current edge**, updating the distance estimate if a shorter path is found.

Iterations

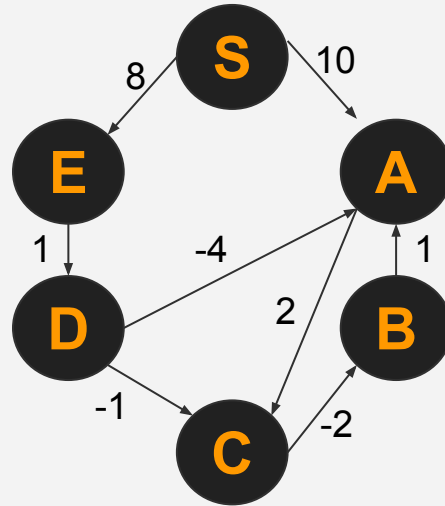
To find the shortest paths in a graph with **N** vertices, the algorithm performs **N-1** edge relaxation iterations. In every iteration we try to minimize the **distance(d)**.

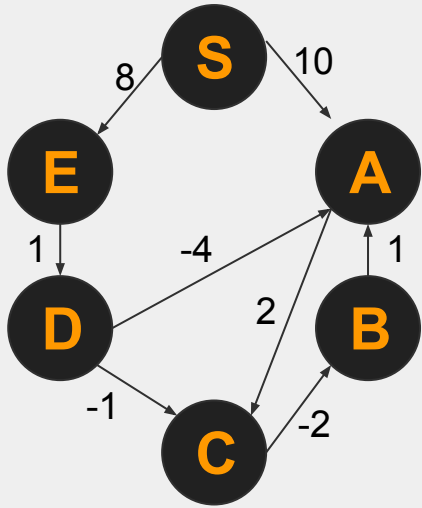
If we have two vertices **u** and **v**:

$$d[v] = \min(d[v], d[u] + d[(u, v)])$$

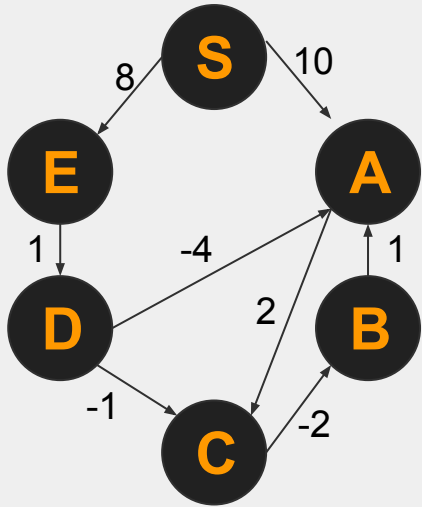
Simulation

What is the shortest path between the source and all other nodes in this graph?

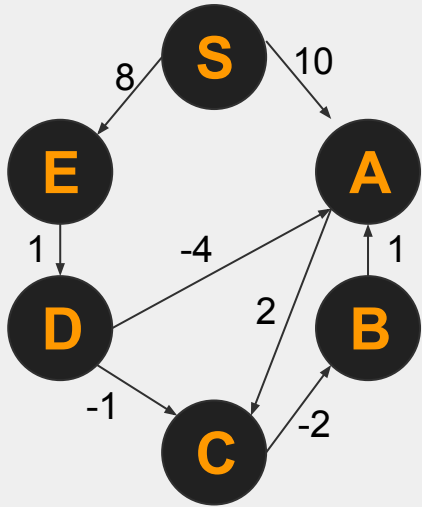




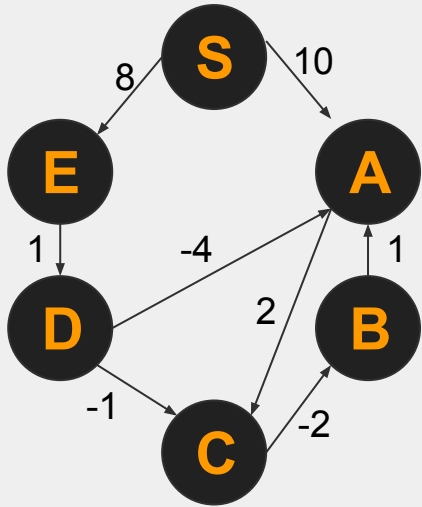
Iteration	S	A	B	C	D	E
initialization	0	inf	inf	inf	inf	inf



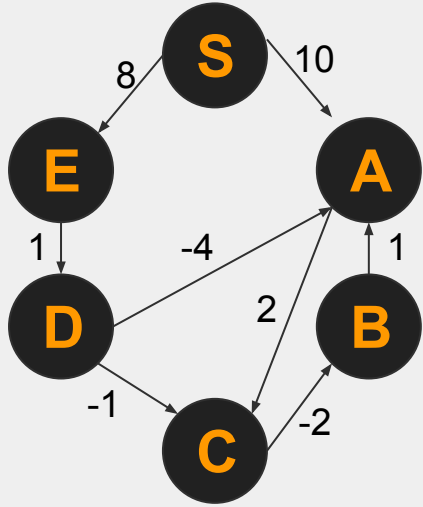
Iteration	S	A	B	C	D	E
initialization	0	inf	inf	inf	inf	inf
1st	0	10	10	12	9	8



Iteration	S	A	B	C	D	E
initialization	0	inf	inf	inf	inf	inf
1st	0	10	10	12	9	8
2nd	0	5	10	8	9	8



Iteration	S	A	B	C	D	E
initialization	0	inf	inf	inf	inf	inf
1st	0	10	10	12	9	8
2nd	0	5	10	8	9	8
3rd	0	5	5	7	9	8



Iteration	S	A	B	C	D	E
initialization	0	inf	inf	inf	inf	inf
1st	0	10	10	12	9	8
2nd	0	5	10	8	9	8
3rd	0	5	5	7	9	8
4th	0	5	5	7	9	8

Should we continue the iteration?

Negative Cycle Detection

The N th iteration is crucial for detecting **negative cycles**, as it aims to find further improvements in distance estimates.

Negative Cycle Detection

During the N th iteration, if the equation $\text{dist}[u] + \text{weight} < \text{dist}[v]$ holds true for any edge (u, v, weight) , it indicates the existence of a **negative cycle**.

If **case 1 fails** for all edges during the N th iteration, there is **no negative cycle** present in the graph.

Handling Disconnected Graphs:

The algorithm assumes that all vertices are **reachable** from the source vertex (usually vertex 0).

To handle disconnected graphs, you can repeat the algorithm for vertices with **distance = INFINITY**.

Complexity

Time complexity of $O(V * E)$ and

Space complexity of $O(V)$, where:

- V represents the number of vertices in the graph.
- E represents the number of edges in the graph.

Bellman-Ford Algorithm Steps

1. Initialize:

Set distances to all vertices to **infinity**, except the **source vertex** (distance = 0).

2. Iterative Relaxation (N-1 Times):

Repeat **N-1** times (N = number of vertices). For each edge (u, v) with weight **w**:

If **distance[u] + w < distance[v]**, update **distance[v]** to **distance[u] + w**.

Bellman-Ford Algorithm Steps

3. Detect Negative Cycles (Nth Iteration):

Repeat relaxation once **more**.

If any distance further reduces, a negative cycle exists.

4. Handle Disconnected Graphs:

Ensure all vertices are visited or repeat for **unvisited vertices**.

5. Output Results:

Retrieve the **shortest distances** from the source vertex to all others.

Implementation

```
def bellman_ford(self, src):  
    # Step 1: Initialize distances  
  
    dist = [float("inf")] * numOfVertices  
  
    dist[src] = 0  
  
    # Step 2: Iterative Relaxation (N-1 times)  
  
    for _ in range(numOfVertices - 1):  
        for u, v, w in self.graph:  
            if dist[u] != float("inf") and dist[u] + w < dist[v]:  
                dist[v] = dist[u] + w
```

Implementation

Step 3: Detect Negative Cycles

```
for u, v, w in self.graph:
    if dist[u] != float("inf") and dist[u] + w < dist[v]:
        print("Negative Cycle Detected")
    return
```

Step 4: Output Results

```
print(dist)
```

Common Pitfalls

Negative cycles: The Floyd-Warshall algorithm assumes that there are no negative cycles in the graph. If a negative cycle exists, the algorithm may not produce correct results.

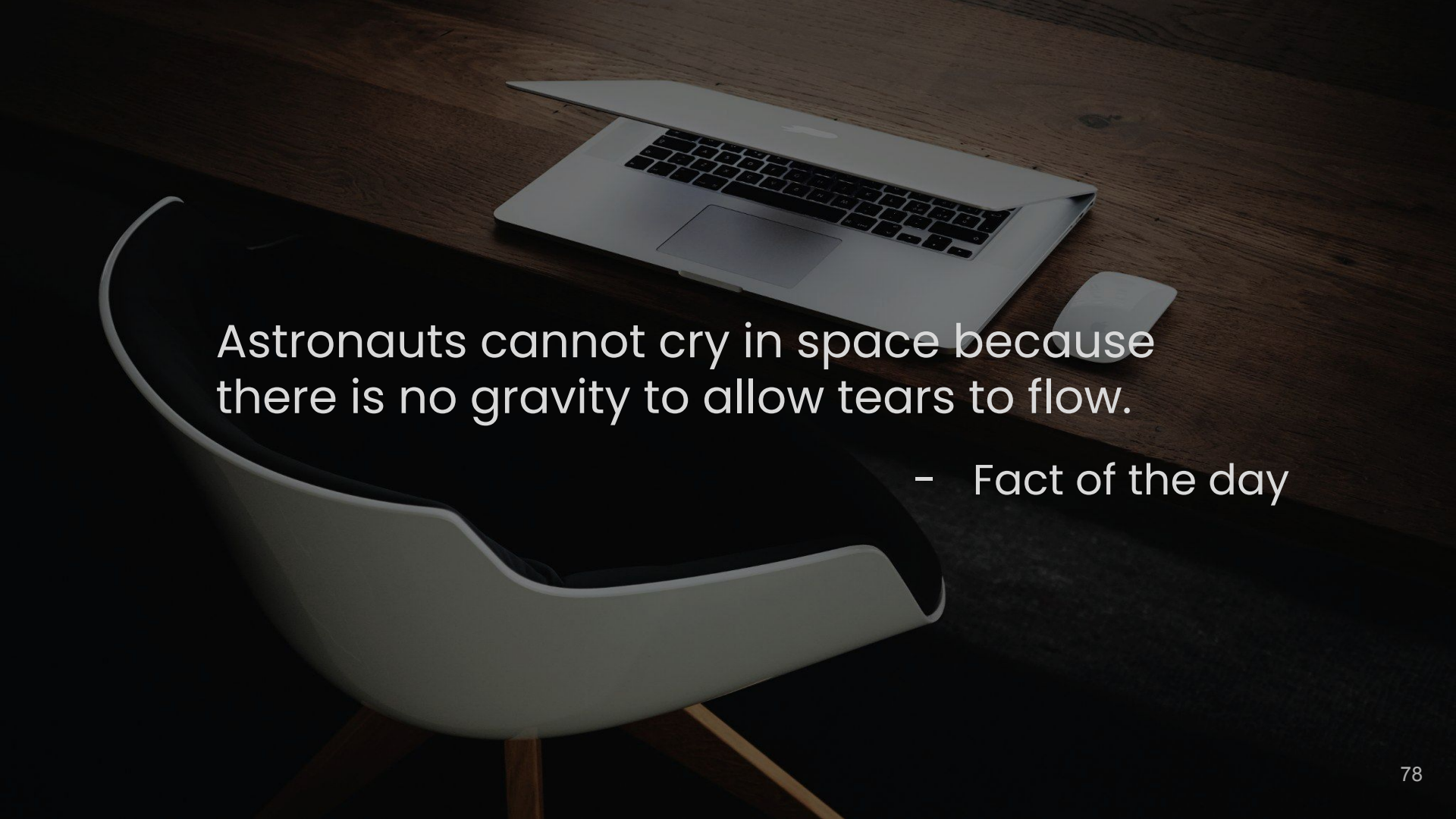
Large graph size: The algorithm's time complexity makes it less suitable for large graphs due to its cubic nature. Consider alternative algorithms like Dijkstra's or Bellman-Ford for specific scenarios.

Practice Questions

- [Path with Maximum Probability](#)
- [Network Delay Time](#)
- [Find the City W](#)
- [Two Routes](#)
- [Course Schedule IV](#)
- [Dijkstra?](#)

Resources

- [USACO GUIDE](#)
- [Abdul Bari - Dijkstra](#)
- [Abdul Bari - Bellman-Ford](#)
- [Abdul Bari - Floyd-Warshall](#)
- [Dev Genius](#)



Astronauts cannot cry in space because there is no gravity to allow tears to flow.

- Fact of the day